

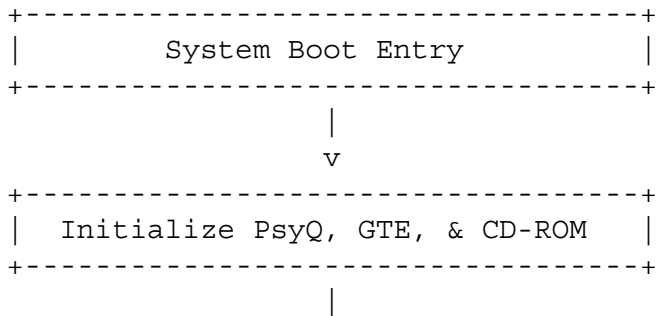
Architectural Analysis of the PlayStation Engine Hybridization in Dragon Quest IV

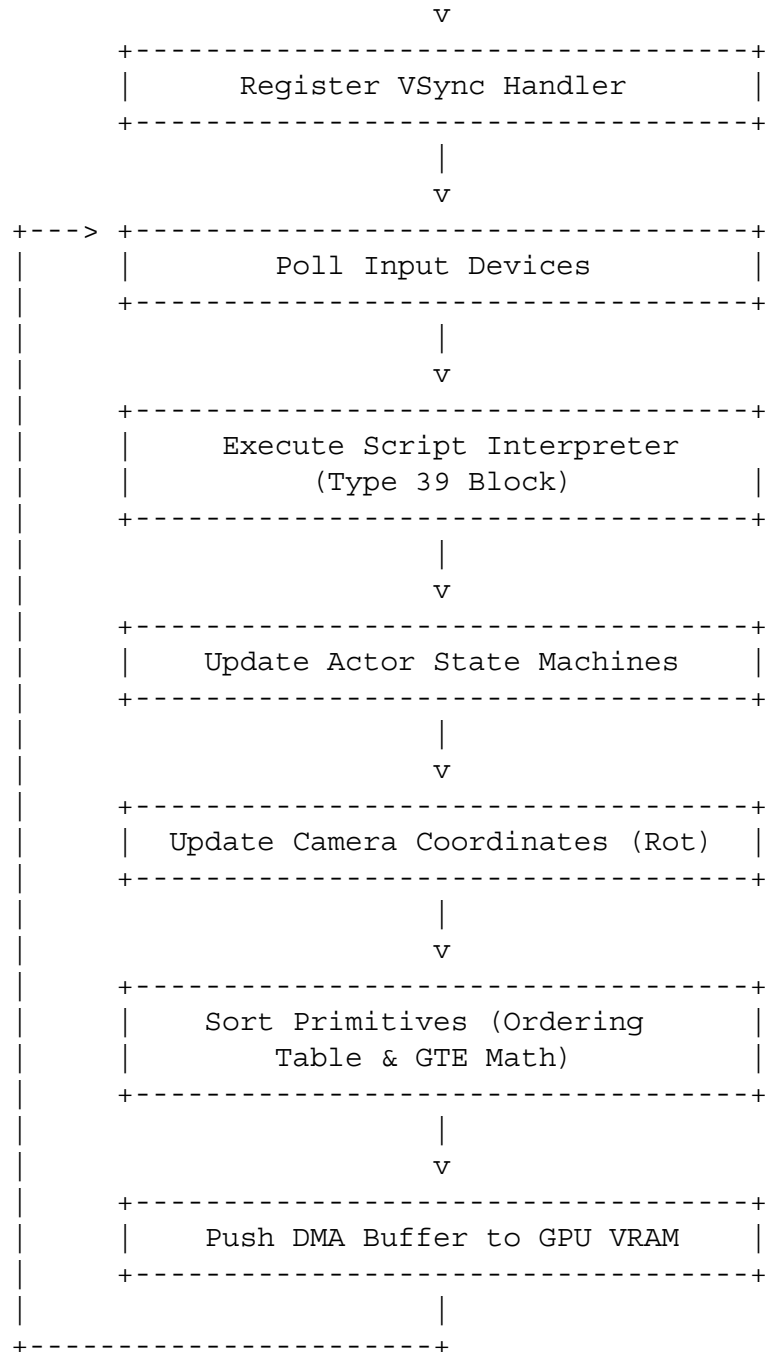
The development of the PlayStation remake of *Dragon Quest IV: Chapters of the Chosen* (SLPM_869.16) represents a sophisticated exercise in software engineering and game engine adaptation. Developed by Heart Beat Inc., the title was constructed by refactoring and adapting the pre-existing, highly complex 3D engine created for *Dragon Quest VII: Eden no Senshitachi* (SLPM_865.00). Heart Beat chose to repurpose this 32-bit codebase to reconstruct a game originally built within the constraints of a 16-bit, tile-based architecture. The technical challenge lay in bridging these distinct design paradigms. The engine had to render a fully rotatable, polygon-based isometric world while maintaining the aesthetic utility of flat, 2D character sprites (billboards) that remained synchronized with a 3D coordinate grid. To accomplish this, Heart Beat injected customized memory limits, optimized asset tables, and engineered a specialized billboard rendering pipeline directly into the core foundation of the *Dragon Quest VII* architecture.

The Executable Bridge: Disassembly and Memory Adaptation

Boot-Stage Execution Flow and Event Loop Registration

Static disassembly of the primary executable for *Dragon Quest IV* (SLPM_869.16) reveals a structured boot-up sequence that establishes the runtime environment before registering the core execution loops. When the PlayStation console boot-rom transfers execution to the system entry point of the executable, the system runs through standard PsyQ compiler initialization libraries. This process configures the coprocessor registers (specifically the Geometry Transfer Engine, or GTE), sets up the stack pointer, and mounts the CD-ROM subsystem. Immediately following the library boot sequence, control is routed to the engine's main initialization function, which starts the primary execution loop. This main event loop is responsible for coordinating the game state, processing inputs, executing the script interpreter, and populating the GPU's primitive drawing lists. It handles frame synchronization by registering a callback routine with the system's vertical blank (VSync) interrupt handler, maintaining a strict frame budget of 30 or 60 fields per second. The event loop acts as a high-level scheduler. In each iteration, it runs the following routine:





BSS Segment Cleansing and Memory Mapping Variations

During this initialization phase, the executable executes a critical memory clearing routine located at 0x8008E284. This function cleanses the BSS (Block Started by Symbol) segment, zeroing out uninitialized global and static variables to prevent garbage data from corrupting the engine state. Comparing this routine and the resulting memory configuration against *Dragon Quest VII* (SLPM_865.00) reveals how the developers modified the memory boundaries to support a hybrid 2D/3D rendering pipeline.

In *Dragon Quest VII*, large blocks of memory were allocated to support the highly detailed 3D structures, multi-layered textures, and complex world map fragments required by its large-scale environment. For *Dragon Quest IV*, Heart Beat optimized these allocations, shrinking the 3D world geometry buffers to free up space for character sprites. This dynamic memory allocation is detailed below:

System Subsystem or Memory Pool	DQVII Memory Allocations (SLPM_865.00)	DQIV Remapped Allocations (SLPM_869.16)	Functional Modification and Engineering Purpose
System Boot Entry Point (start)	0x8008DAC0	0x8008DAC0	Unchanged standard compiler entry point for hardware initialization.
BSS Clearing Routine	0x8008DAC0 - 0x8008E1F0	0x8008E284 (Entry Target)	Shifted offset to accommodate customized engine initializers.
BGM Sequence Queue (Dynamic Buffer)	0x800D25C0 - 0x800D80EF	0x800D1500 - 0x800D7500	Reduced footprint to expand main RAM variable storage.
Waveform Instrument Attributes	0x800D80F0 - 0x800DA0F7	0x800D7600 - 0x800D9600	Relocated to align with the customized sound driver footprint.
VRAM Draw/Display Double Buffers	0x800B9D10 - 0x800BB6E7	0x800BA800 - 0x800BC200	Expanded page buffers to prevent frame dropping during map rotation.
Sound Interrupt Thread	0x8002FA10 - 0x80036FCC	0x8002FA10 - 0x80036FCC	Preserved core driver architecture for sound playback.
	Sequence Interpretation Tables	0x80025300 - 0x800258E8	0x80025300 - 0x800258E8
By shifting the BSS boundaries, Heart Beat expanded the maximum allocation size for the sprite lookup tables. Unlike <i>Dragon Quest VII</i>, which used simple 3D models for many secondary characters, <i>Dragon Quest IV</i> retained classic 2D character designs, requiring constant buffering of multi-directional, highly			

System Subsystem or Memory Pool	DQVII Memory Allocations (SLPM_865.00)	DQIV Remapped Allocations (SLPM_869.16)	Functional Modification and Engineering Purpose
detailed sprite sheets for players and NPCs.			

Sprite-to-3D-Grid Mapping Routine Initialization

The specific routine responsible for initializing the mapping of 2D sprites onto the 3D isometric grid is located at offset 0x8005C1F0 in the program memory of SLPM_869.16. This routine runs immediately after the engine's rendering context is established. It registers a structured array of actor state blocks in memory, where each entry contains a 3D position vector, a 2D viewport anchor point, an active rotation angle, and a pointer to the character's sprite sheet. This initialization routine sets up coordinate transformation registers on the PlayStation's GTE, defining how the 3D coordinate frame of the terrain grid translates to the screen space coordinates of the billboard quads. By configuring these coordinate registers during the boot phase, the engine ensures that every entity loaded by the map loader is correctly positioned relative to the underlying 3D world geometry, preventing visual alignment issues when the scene is rendered.

Asset Table Mapping and the LBA File Structure

Archive Format and Sector-Level Layout

All non-executable assets for *Dragon Quest IV* are packed into a single, massive resource archive named HBD1PS1D.Q41. This file is structured in a format identical to *Dragon Quest VII*'s archive, HBD1PS1D.Q71. The archive spans exactly 319,436,800 bytes, dividing evenly into 155,975 blocks of 2048 bytes, which matches the standard physical sector size of a CD-ROM.

The first block (LBA index 0, offsets [span_24](start_span)[span_24](end_span)0x00 through 0x800) holds the volume identification header, containing the ASCII identifier hdb1ps1d.q41 at position 0x400 (1024). The remaining sectors follow a strictly aligned block system, beginning with an integer value that defines the sector payload.

+----- [span_26] (start_span) [span_26] (end_span) -----+	
	HBD1PS1D.Q41 Sector
+-----+	
	0x00 Main Block Header (16 Bytes)
	- Sub-block count (uint32)
	- Sector size of block (uint32)
	- Total decompressed raw data length (uint32)
	- Reserved / zero padding (uint32)
+-----+	
	0x10 Sub-Block Headers (16 Bytes per Sub-block)
	- Compressed data size (uint32)
	- Decompressed data size (uint32)
	- Offset within sector payload (uint32)
	- Compression Flag (uint16) [e.g., 1280]

- Sub-Block Type (uint16)

Raw Payload Segment
- Compressed or raw data blocks
- Padded with 0x00 to complete 2048-byte boundary

The compression flag inside the sub-block header (offset 0x0c) typically contains the decimal value 1280 (or 0x0500), which indicates that the payload is compressed using Heart Beat's custom variant of the Lempel-Ziv-Storer-Szymanski (LZS) algorithm. If the block is uncompressed, this flag is set to 0, and the compressed data size matches the decompressed data size.

The sub-block type at offset 0x0e categorizes the resource payload, telling the engine's loader which subsystem should receive the data. The sub-block types are mapped as follows:

Sub-Block Type Code	Compression Status	Primary Resource Contents	Role and System Integration
Type 1	Uncompressed	Font Glyph Sheets	Loaded directly into VRAM to populate the text engine's character cache.
Type 6	Compressed	Map Chipset Images	Holds textures and 2D tilesets used to construct world and town maps.
Type 8	Compressed	Monster & Battle Sprites	Standard PlayStation TIM images detailing battle animations and opponent sprites.
Type 10	Compressed	Multi-TIM Sprite Packages	Multi-directional frames for active combat graphics and effects.
Type 13	Compressed	NPC & Player Sprites	Multi-directional sheets containing player character and town NPC animation cycles.
Type 21	Uncompressed	3D Map Geometry (qQES)	Holds the underlying 3D world meshes, polygon definitions, and collision fields.
Type 32	Uncompressed	Scene Dialogue Index Maps	Translates active script commands to physical text offsets in dialogue files.
Type 39	Compressed	Primary Dialogue Script Engine	Contains the script instructions and text data.

Dialogue Script Parsing and Conditional Control Structures

Text Block Architecture and Parser State

Dialogue files (Type 39 sub-blocks) are compressed using Huffman coding. When a dialogue sequence is triggered, the engine locates the associated script segment using an offset index from Type 32 blocks. The script block starts with a structured header that directs the parsing engine:

+-----+ Text Block Header +-----+	
Offset 0x00: Pointer offset "a" to end of block payload (uint32)	
Offset 0x04: Unique Scenario Scene ID (uint32)	
Offset 0x08: Offset pointer "c" to Huffman-encoded bitstream	
Offset 0x0C: Offset pointer "d" to Huffman Tree structures	
Offset 0x10: Offset pointer "e" to end of Huffman bitstream	
Offset 0x14: Reserved / padding	
+-----+	

The script execution path is driven by a specialized virtual machine command, C021A0. This instruction acts as the primary program controller for the dialogue engine:

- C021A0 <offset> <dialogId>: Directs the interpreter to load a specific dialogue stream, jumping to the designated offset inside the decompressed text buffer.
- C[span_64](start_span)[span_64](end_span)021A0 <FFF0> <key>: Evaluates event flags or registers system variables (such as item counts or active party members) to alter the dialogue flow.

Formatting and Parser Control Codes

The decoded text stream contains inline control characters that handle window formatting, variable placement, and parser state :

- 0000: Closes the current dialogue stream and terminates the window thread.
- 7[span_67](start_span)[span_67](end_span)f02: Inserts a newline and tab, resetting the text cursor to the left margin of the next dialogue line.
- 7[span_68](start_span)[span_68](end_span)f04 / 7fXX: Name decorator; tells the interpreter to look at the next byte (7fXX) and render the matching actor's name.
- 7[span_69](start_span)[span_69](end_span)f0a / 7f0b: Standard end-of-line signifiers that display a blinking cursor and pause the script engine until player input is received.
- 7[span_70](start_span)[span_70](end_span)f15: System variable token that dynamically inserts numeric values, such as gold counts, into the text stream.

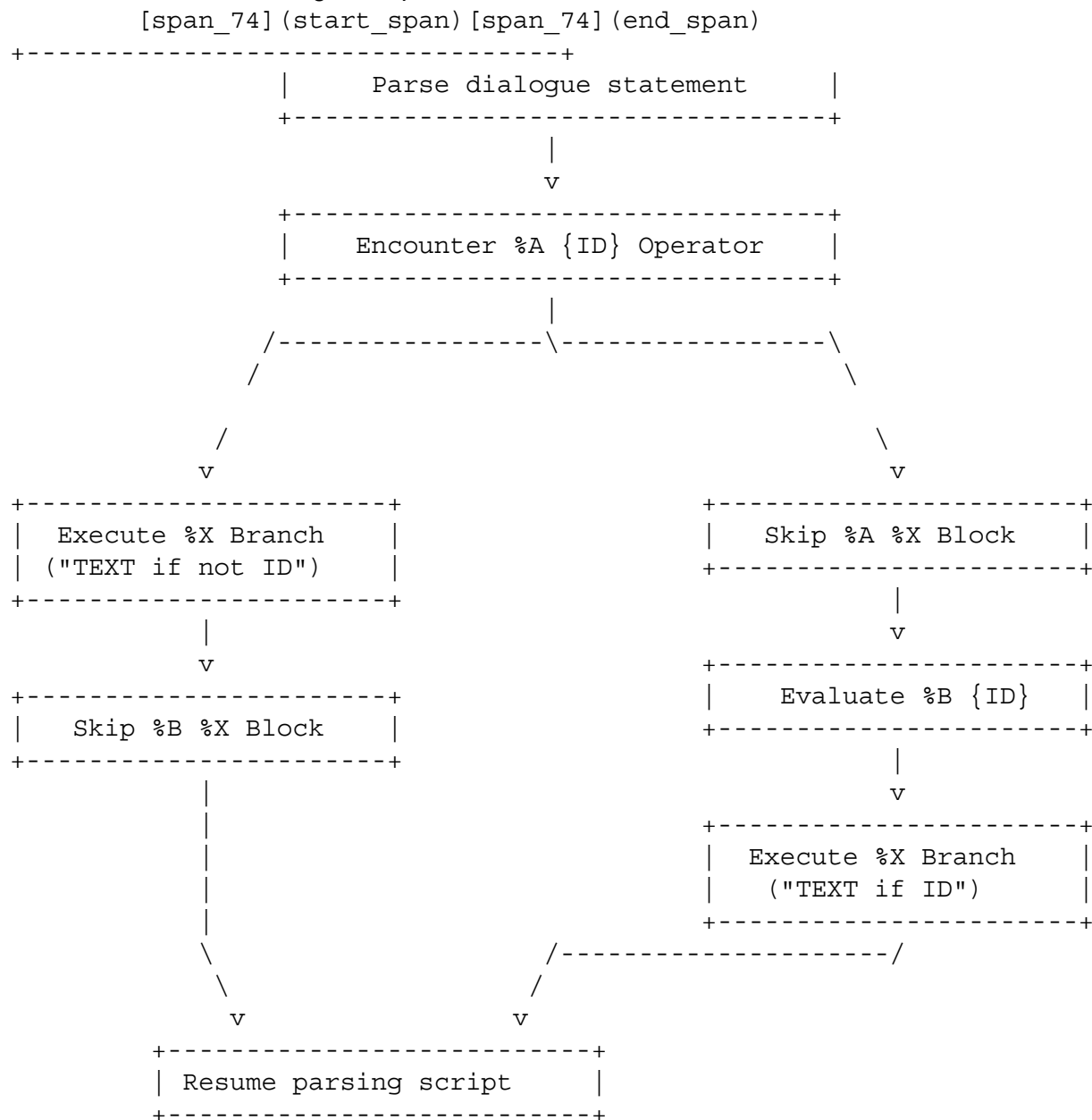
Conditional Scripting Syntax and Multi-Chapter Execution

To manage the game's chapter-based progression—where different characters lead the active party depending on the current narrative phase—Heart Beat implemented a robust conditional

branching syntax directly into the script interpreter :

`\%A{\text{ID}}\}%X{\text{TEXT if not ID}}\}%Z%B{\text{ID}}\}%X{\text{TEXT if ID}}\}%Z`

When the interpreter encounters the %A control statement, it evaluates the subsequent character ID against the current party leader's ID. If the active leader does *not* match the specified ID, the engine executes the %X text block until it hits the %Z terminator. If the active leader *does* match, the engine skips the %A block and executes the %B branch instead.



This conditional syntax allows the engine to adapt dialogue to the active chapter's perspective without duplicating script assets. For example, when Alena (ID 120) is the active party leader, the parser dynamically shifts from a third-person narrative to a direct second-person response:

- **Dialogue Stream:** %A120%XStill, if she were a boy—%Z%B120%XStill, if you were a boy—%Z
- **Result (Active leader is Alena):** "Still, if you were a boy—"

- **Result (Active leader is Ragnar or Torneko):** "Still, if she were a boy—"

For plurals and variable numeric values, the script engine uses a specialized conditional operator :

$\backslash\%H\{\textrm{span_78}(start_span)[span_78](end_span)t\{Variable\ ID\}\}\%X\%Ys\%Z$

This operator checks the state of the designated numeric variable (e.g., a gold coin count). If the variable's value is greater than 1, the script engine renders the plural suffix contained within the %Y block, ensuring correct grammar across all dialogue text.

Sprite-Grid Synchronization and VRAM Buffer Processing during Map Rotation

Billboard Coordinate Transformation Mathematics

To construct the world map, the engine merges a fully rotatable 3D polygonal landscape with flat, 2D character sprites. The player can rotate the camera viewpoint in 90-degree increments (with smooth interpolation transitions). During this rotation, character sprites must remain flat and face the screen directly to avoid rendering anomalies or looking like flat sheets.

To achieve this, the coordinate processing engine splits its transformation pipeline into two distinct pathways: one for the 3D grid tiles, and another for the 2D billboards.

Let the coordinate of a 3D grid vertex in world space be defined as:

$\mathbf{P}_w = \begin{pmatrix} X_w \\ Y_w \\ Z_w \end{pmatrix}$

The camera's orientation is defined by a rotation matrix $\mathbf{R}_y(\theta)$ around the vertical Y-axis, where θ represents the current camera rotation angle:

$\mathbf{R}_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}$

Applying the camera's translation vector $\mathbf{T} = {}^A\mathbf{T}$, the 3D grid vertex is transformed into camera-space coordinates $\mathbf{P}_c$:

$\mathbf{P}_c = \mathbf{R}_y(\theta) \mathbf{P}_w + \mathbf{T}$

For 2D character sprites, applying this transformation directly would rotate the flat sprite quad relative to the screen, causing it to flatten and disappear as the angle approached 90 degrees.

To prevent this, the engine uses a specialized transformation path for billboard sprites:

1. The engine extracts the sprite's 3D anchor position in world space, $\mathbf{A}_w = [A_x, A_y, A_z]^T$.
2. The anchor point is transformed into camera space using the standard camera transformation, determining the sprite's correct depth and position relative to the camera:
 $\mathbf{A}_c = \mathbf{R}_y(\theta) \mathbf{A}_w + \mathbf{T}$
1. To generate the four vertices of the sprite quad $\mathbf{V}_c^{(i)}$ (where $i \in \{1, 2, 3, 4\}$), the coordinate processing function adds the local width (w) and height (h) offsets of the sprite directly to the camera-space anchor point $\mathbf{A}_c$. This step bypasses the camera's rotation matrix for the sprite's internal geometry:

$\mathbf{V}_c^{(i)} = \mathbf{A}_c + \begin{pmatrix} \Delta x^{(i)} \\ \Delta y^{(i)} \\ 0 \end{pmatrix}$

Here, $\Delta x^{(i)} \in \{-w/2, w/2\}$ and $\Delta y^{(i)} \in \{0, h\}$, depending on the specific corner of the quad being calculated. Because the Z-displacement in camera space remains zero relative to the anchor point ($\Delta z = 0$), the entire sprite quad remains perfectly parallel to the screen's projection plane.

Finally, the PlayStation's GTE projects these camera-space coordinates to 2D screen

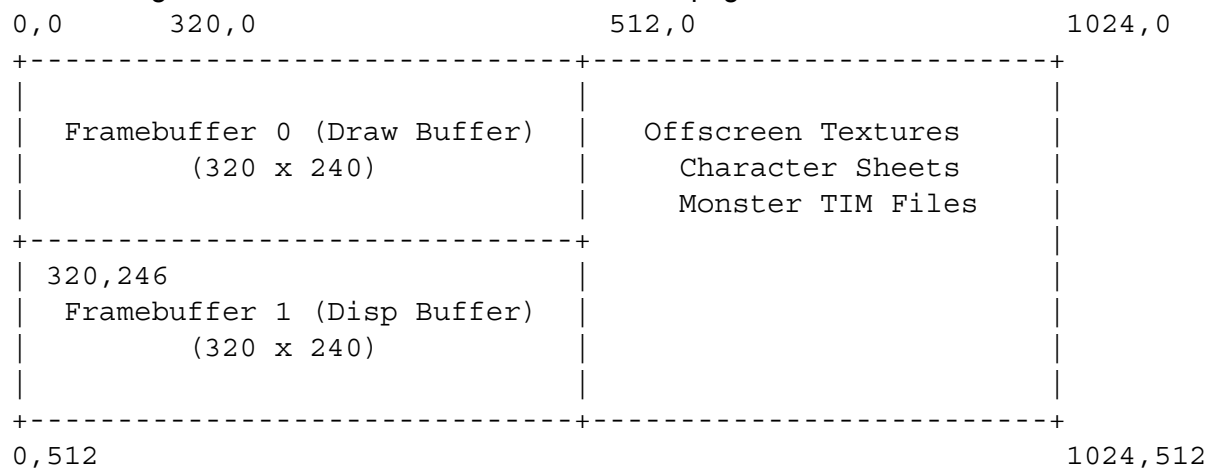
coordinates (X_s, Y_s) using perspective projection:

$$X_s = \frac{V_{c,x} \cdot f}{V_{c,z}} + X_0 \quad Y_s = \frac{V_{c,y} \cdot f}{V_{c,z}} + Y_0$$

where f represents the camera's focal length and (X_0, Y_0) represents the projection center offset in the framebuffer. This dual-path transformation system ensures that character sprites remain upright and face the player during active map rotations.

VRAM Layout and Page Swapping

To prevent screen tearing and rendering artifacts when the camera is rotating, Heart Beat's engine utilizes double-buffered page swapping. The 1024x512 pixel VRAM space is divided into two drawing buffers and dedicated offscreen texture pages:



The active drawing area is divided into two buffers: Framebuffer 0 (at 0, 0 to 320, 240) and Framebuffer 1 (at 0, 256 to 320, 496). Sprite sheets for players and NPCs (sub-block types 13, 14, and 15) are cached in the offscreen texture area located above coordinate X=512. Color Lookup Tables (CLUTs) are stored in the GPU's dedicated palette memory, mapping 4-bit or 8-bit index values to 16-bit high-color values.

During map rotation, the coordinate processor transforms the 3D terrain grid and determines the screen-space coordinates of the character sprite quads. It then writes these primitives to the GPU's Ordering Table (OT) in back-to-front order. This ensures correct depth sorting, drawing the character sprites over the background terrain tiles while preventing visual clipping during camera movement.

Works cited

1. Dragon „Hackst“ IV | Markus Projects, <http://markus-projects.net/dragon-hackst-iv/>
2. Dragon Quest IV: Michibikareshi Monotachi • ドラゴンクエストIV 導かれし者たち - Redump.org, <http://redump.org/disc/3867/>
3. Mednafen-Core/MednafenGameCore.mm at master - GitHub, <https://github.com/OpenEmu/Mednafen-Core/blob/master/MednafenGameCore.mm>
4. Dragon Quest VII: Sound Engine Analysis - GitHub Gist, <https://gist.github.com/loveemu/582ff7683a797b7048b6>
5. DRAGON QUEST IV - Ratings & Reviews - App Store, <https://apps.apple.com/us/app/dragon-quest-iv/id847381935?see-all=reviews&platform=iphone>
6. Dragon Quest IV - Wikipedia, https://en.wikipedia.org/wiki/Dragon_Quest_IV
7. Dragon Quest VII (aka Dragon Warrior VII) Review | RPG's Suck - WordPress.com,

<https://badrpgreviews.wordpress.com/2015/01/19/dragon-quest-vii-review/> 8. Opinions on HD2D? Surely it's not the only option. : r/chronotrigger - Reddit, https://www.reddit.com/r/chronotrigger/comments/1jl6v2m/opinions_on_hd2d_surely_its_not_the_only_option/ 9. Dragon Warrior VII - Guide and Walkthrough - PlayStation - By stahlbaum - GameFAQs, <https://gamefaqs.gamespot.com/ps/197152-dragon-warrior-vii/faqs/54482> 10. Dragon Quest IV 4 PS1 PlayStation 1 Japan Import US Seller SLPM-86916 | eBay, <https://www.ebay.com/itm/275821319629> 11. Dragon Quest VII: Fragments of the Forgotten Past Review - GameSpew, <https://www.gamespew.com/2016/09/dragon-quest-vii-fragments-of-the-forgotten-past-review/> 12. Dragon Quest VII: Fragments of the Forgotten Past, https://dragon-quest.org/wiki/Dragon_Quest_VII:_Fragments_of_the_Forgotten_Past 13. Attempt at Python tooling for the Dragon Quest 4 PSX Translation - GitHub, <https://github.com/mwilkins/dq4psxtrans>